

Universidad Mariano Gálvez

[Nombre del curso]

Análisis, Rediseño Arquitectónico y Contrato API

Proyecto FitControl

Integrantes: [Nombres del equipo]

Docente: [Nombre del docente]

Fecha: [dd/mm/aaaa]

1. Introducción

Este informe analiza FitControl, un sistema de escritorio para la administración de un gimnasio (control de clientes, planes, membresías y reportes), construido en C# (Windows Forms) con una base de datos SQL Server. El propósito del proyecto es transformar este sistema monolítico y cerrado en una plataforma que exponga sus funcionalidades mediante una API REST, permitiendo que aplicaciones externas —un portal web, una app móvil y una pasarela de pago— puedan integrarse sin depender del cliente de escritorio original.

El problema que se pretende resolver es doble: por un lado, el acoplamiento actual entre interfaz, lógica de negocio y acceso a datos hace que el sistema sea difícil de mantener y de extender; por otro, la arquitectura actual no contempla ningún mecanismo de acceso externo ni de procesamiento de pagos en línea, funcionalidades que el negocio necesita hoy.

Como solución, se propone migrar hacia un Monolito Modular: un único backend desplegable, pero internamente organizado en módulos independientes (Autenticación, Clientes, Membresías, Pagos y Reportes), expuesto a través de una API REST documentada en OpenAPI 3.0, con una base de datos relacional normalizada y una capa de caché (Redis) para las consultas más frecuentes.

2. Contexto del sistema legado

2.1 Descripción del sistema actual

FitControl tiene como propósito administrar las operaciones diarias de un gimnasio: registro de clientes, control de planes y membresías, y generación de reportes administrativos. Sus usuarios principales son la recepcionista (registra clientes y membresías) y el administrador (consulta reportes y gestiona el sistema). Los procesos de negocio cubiertos son: autenticación de personal, alta y edición de clientes, definición de planes, activación y seguimiento de membresías, y reportes de ingresos y vencimientos.

La arquitectura actual es un monolito de escritorio sin separación de capas: cada formulario de Windows Forms (FrmLogin, FrmClientes, FrmMembresías, FrmPlanes, FrmReportes) contiene en su propio código la interfaz, la lógica de negocio y las sentencias SQL que se ejecutan contra la base de datos, a través de una única clase de conexión (conexion.cs) basada en ADO.NET.

2.2 Tecnologías actuales

Elemento	Tecnología
Lenguaje	C# (.NET Framework)
Framework	Windows Forms
Base de datos	SQL Server Express
Frontend	No aplica (cliente de escritorio nativo)
Infraestructura	Instalación local / red interna, sin servidor de aplicaciones
Integraciones	Ninguna (sin acceso externo ni pasarela de pago)

2.3 Puntos de dolor identificados

Problema	Causa	Consecuencia	Evidencia
Acoplamiento	UI, lógica y SQL mezclados en cada formulario	Imposible reutilizar la lógica en una API o hacer pruebas unitarias	FrmClientes.cs, FrmMembresias.cs

Seguridad	Credenciales de BD y contraseñas de usuario en texto plano	Cualquiera con el código fuente obtiene acceso total a la base de datos	conexion.cs
Escalabilidad	Cada cliente se conecta directo a una única instancia de SQL Server	No soporta acceso concurrente de apps externas ni picos de tráfico	Arquitectura cliente-servidor directa
Disponibilidad	Un solo punto de falla (una instancia de BD, sin caché ni réplica)	Una caída de la BD detiene toda la operación	Sin mecanismo de respaldo automático observado
Mantenibilidad	Lógica de negocio dispersa en el code-behind de la UI	Cambios pequeños requieren tocar múltiples formularios sin pruebas	Ausencia de capa de servicios/repositorios

3. Ingeniería inversa del sistema actual

3.1 Análisis de la arquitectura existente

El sistema actual funciona como una aplicación de escritorio monolítica: al iniciar, FrmLogin valida usuario y contraseña directamente contra la tabla Usuarios. Una vez autenticado, el usuario navega entre formularios (FrmClientes, FrmMembresias, FrmPlanes, FrmReportes), cada uno de los cuales construye sus propias consultas SQL y las ejecuta contra la base de datos usando la conexión centralizada en conexion.cs. No existe una capa de servicios, repositorios ni controladores: la interfaz gráfica es, al mismo tiempo, la capa de negocio y de acceso a datos.

3.2 Diagrama del sistema legado

El siguiente diagrama de componentes representa la arquitectura actual: todos los formularios están agrupados dentro de la aplicación de escritorio, sin separación de capas, y se conectan directamente a la base de datos.

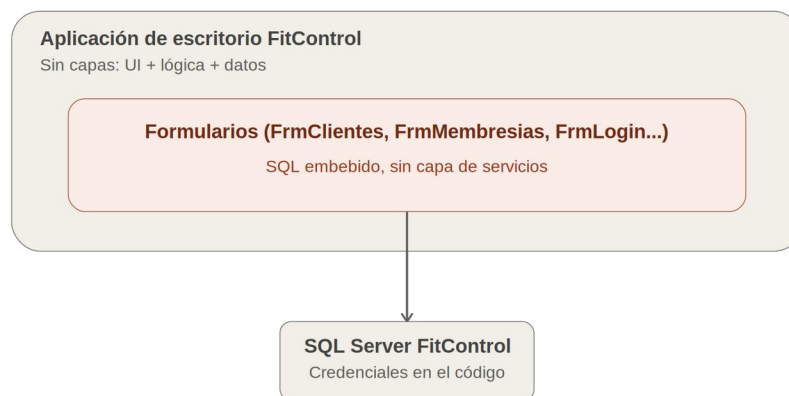


Diagrama 1 — Arquitectura del sistema legado

Archivo: docs/legacy/arquitectura_legado.png

Fuente editable: docs/legacy/arquitectura_legado.puml (o equivalente)

4. Análisis de alternativas arquitectónicas

4.1 Alternativas consideradas

- Monolito actual: mantener la arquitectura de escritorio sin cambios.
- Microservicios: dividir el sistema en servicios independientes por dominio (Clientes, Membresías, Pagos, Reportes), cada uno con su propia base de datos y despliegue.
- Monolito modular: un único backend desplegable, organizado internamente en módulos independientes con interfaces claras entre ellos.
- Event-Driven Architecture: los módulos se comunican de forma asíncrona mediante eventos (por ejemplo, "membresía creada", "pago aprobado") a través de un bus de mensajes.

4.2 Matriz comparativa

Criterio	Monolito	Microservicios	Monolito modular	Event-driven
Escalabilidad	Baja — un solo cliente/BD	Alta, pero sobredimensionada para este caso	Media-alta, suficiente para el volumen de un gimnasio	Alta, pero con complejidad innecesaria
Mantenibilidad	Baja — todo mezclado	Alta, pero multiplica repos y despliegues	Alta — módulos separados en un solo despliegue	Media — lógica dispersa en eventos
Disponibilidad	Baja — un solo punto de falla	Alta (resiliencia por servicio)	Media-alta	Alta
Seguridad	Baja — credenciales expuestas	Alta si se implementa bien; más superficie de ataque	Alta — punto de entrada único con JWT centralizado	Media — depende del bus de eventos
Complejidad / costo	Baja (statu quo)	Alta (orquestación, red, DevOps)	Baja-media	Alta (broker, consistencia eventual)
Riesgo para el proyecto	Alto (statu quo insostenible)	Alto (sobreingeniería)	Bajo	Medio-alto

4.3 Arquitectura seleccionada

Arquitectura propuesta: Monolito Modular

4.4 Justificación técnica

- **Escalabilidad:** al separar el backend en módulos internos (Auth, Clientes, Membresías, Pagos, Reportes) comunicados por interfaces claras, el proceso completo puede escalar horizontalmente (varias instancias detrás de un balanceador) sin la complejidad operativa de microservicios, que no se justifica para el volumen de un solo gimnasio.
- **Mantenibilidad:** cada módulo tiene su propia capa de controlador, servicio y repositorio, eliminando el acoplamiento actual entre UI y SQL, y permitiendo pruebas unitarias y cambios aislados por módulo.
- **Disponibilidad:** al desacoplar la lógica de la base de datos mediante repositorios, se puede introducir caché (Redis) y reintentos sin tocar la lógica de negocio, reduciendo el impacto de una caída puntual de la base de datos.
- **Seguridad:** un único punto de entrada (la API) permite centralizar la autenticación JWT y elimina el problema actual de credenciales embebidas en cada cliente de escritorio.

5. Contrato API — OpenAPI 3.0

5.1 Descripción de la API

Campo	Detalle
Nombre	FitControl API
Propósito	Exponer la gestión de clientes, membresías, planes, pagos y reportes del gimnasio a consumidores externos
Versión	v1.0 (openapi: 3.0.3)
Recursos principales	/auth, /clientes, /planes, /membresias, /payments, /reportes
Consumidores	App de escritorio actual, futura app web/móvil, pasarela de pago

5.2 Endpoints

Método	Endpoint	Operación	Autenticación
POST	/auth/login	Autentica y devuelve un token JWT	No requiere
GET	/clientes	Lista los clientes registrados	Bearer JWT
POST	/clientes	Registra un nuevo cliente	Bearer JWT
PUT	/clientes/{id}	Actualiza los datos de un cliente	Bearer JWT
POST	/membresias	Registra una nueva membresía	Bearer JWT
POST	/payments/process	Procesa el pago de una membresía	Bearer JWT

5.3 – 5.5 Detalle de endpoints, DTOs y códigos HTTP

El detalle completo de cada endpoint —parámetros, request body, DTO utilizado, respuesta y códigos HTTP— está documentado en el archivo OpenAPI 3.0 adjunto, incluyendo los esquemas LoginRequest, LoginResponse, PaymentRequest, PaymentResponse y ErrorResponse bajo components/schemas. Se estandarizaron los códigos 200 (OK), 201 (Created), 400 (Bad Request), 401 (Unauthorized) y 404 (Not Found), todos con el mismo formato de respuesta: { success, data } en éxito y { success: false, error: { code, message } } en error.

5.6 Seguridad

La API utiliza autenticación Bearer con JWT: el endpoint /auth/login entrega un token firmado con expiración (expiresIn). Todos los endpoints protegidos (por ejemplo /payments/process) exigen el encabezado Authorization: Bearer <token>, validado por un middleware antes de llegar al controlador. Los endpoints están protegidos según el esquema de seguridad bearerAuth definido en components/securitySchemes.

Evidencias del contrato API

Especificación OpenAPI: docs/api/openapi.yaml

Swagger UI: docs/api/swagger_ui.png

La evidencia debe mostrar el contrato cargado en editor.swagger.io o en una instancia de Swagger UI, confirmando que no hay errores de sintaxis y que los endpoints, DTOs y respuestas se visualizan correctamente.

6. Diseño de arquitectura C4

6.1 Nivel 1 — System Context

Muestra a los tres actores del sistema (Cliente, Recepcionista, Administrador) interactuando con el Sistema FitControl, el cual a su vez se conecta con un único sistema externo: la Pasarela de Pago, que procesa los cobros de membresías.

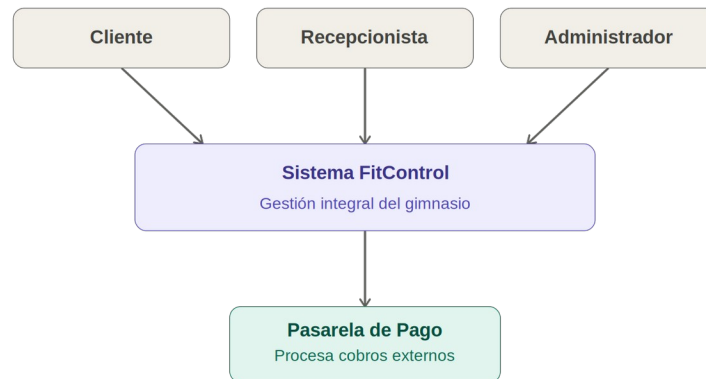


Diagrama 2 — C4 Nivel 1: Contexto

Archivo: docs/c4/c4_nivel1_contexto.png

Fuente editable: docs/c4/src/c4_nivel1_contexto.puml

6.2 Nivel 2 — Container Diagram

Dentro del límite del Sistema FitControl se identifican los contenedores desplegables: la App de escritorio actual (WinForms), la futura App Web/Mobile, el API Backend (que expone los servicios REST), la Base de datos (SQL Server) y la Caché (Redis). Fuera del límite se mantiene la Pasarela de Pago como servicio de terceros.

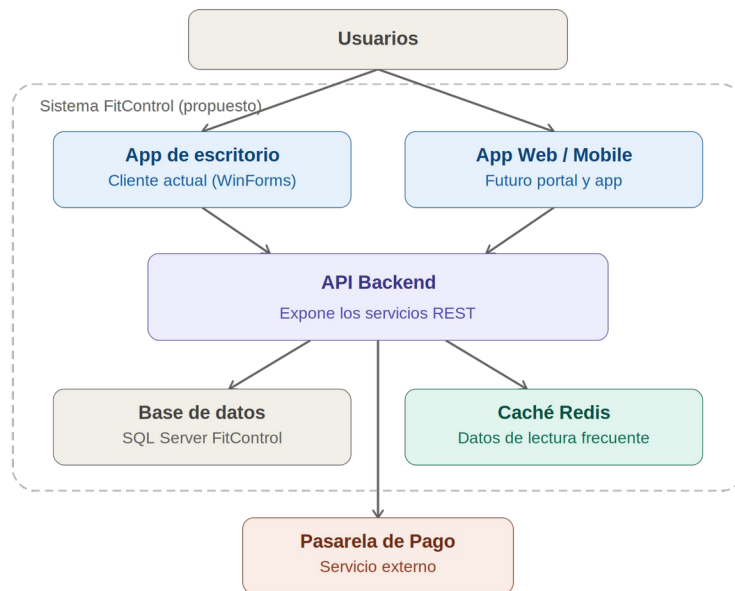


Diagrama 3 — C4 Nivel 2: Contenedores

Archivo: docs/c4/c4_nivel2_contenedores.png

Fuente editable: docs/c4/src/c4_nivel2_contenedores.puml

6.3 Nivel 3 — Component Diagram

Dentro del contenedor API Backend se identifican sus componentes internos: un Middleware JWT que valida el token en cada petición, los Controllers (Auth, Clientes, Membresías, Pagos), los Services con la lógica de negocio, los Repositories que acceden a los datos, y los DTOs que definen los contratos de entrada y salida entre estas capas.

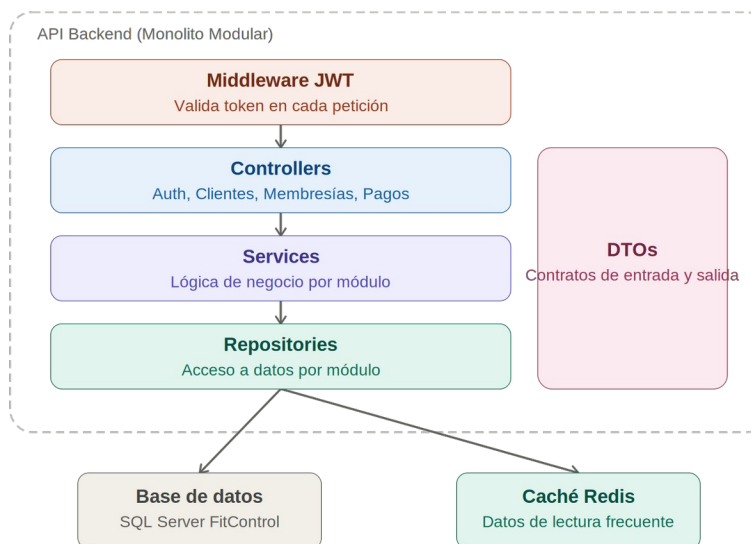


Diagrama 4 — C4 Nivel 3: Componentes

Archivo: docs/c4/c4_nivel3_componentes.png

Fuente editable: docs/c4/src/c4_nivel3_componentes.puml

6.4 Nivel 4 — Code Diagram

El detalle a nivel de código (diagrama entidad-relación, clases y secuencia) se documenta en las secciones 7 y 9 de este informe, correspondientes al modelo de datos y a la estrategia de caché.

7. Diseño de persistencia de datos

7.1 Tecnología seleccionada

Se mantiene SQL Server como motor relacional, dado que ya es la tecnología del sistema legado y el dominio (clientes, planes, membresías, pagos) es fuertemente relacional, con relaciones de integridad claras entre entidades (una membresía siempre pertenece a un cliente y a un plan; un pago siempre pertenece a una membresía).

7.2 Modelo de datos

El modelo agrega una entidad que el sistema legado no contemplaba: Pagos, necesaria para registrar las transacciones que procesa el endpoint /payments/process. Las relaciones principales son: un Cliente puede tener muchas Membresías; un Plan puede estar incluido en muchas Membresías; y una Membresía puede generar uno o varios Pagos.

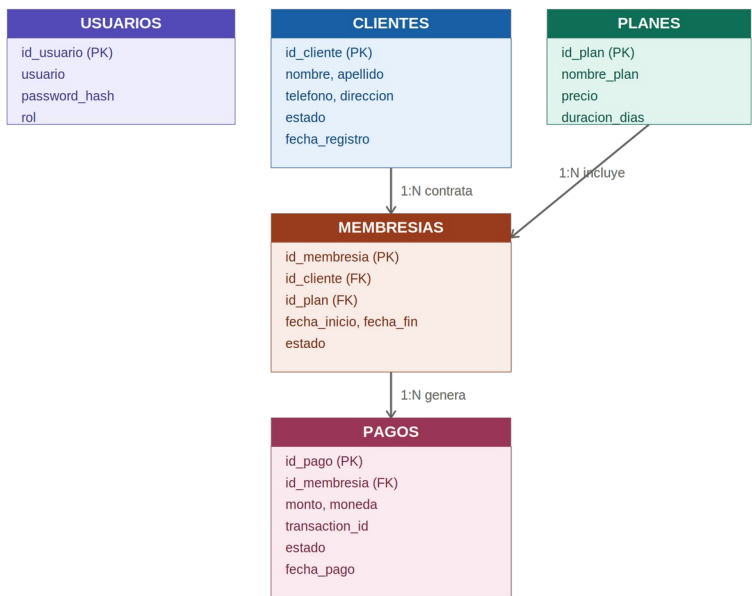


Diagrama 5 — Modelo de datos (DER)

Archivo: docs/database/modelo_datos.png

7.3 Entidades principales

Entidad	Responsabilidad	Atributos principales
Usuarios	Personal que inicia sesión en el sistema	id_usuario, usuario, password_hash, rol
Clientes	Afiliados del gimnasio	id_cliente, nombre, apellido, telefono, direccion, estado
Planes	Catálogo de planes ofrecidos	id_plan, nombre_plan, precio, duracion_dias
Membresias	Vincula un cliente con un plan y su vigencia	id_membresia, id_cliente, id_plan, fecha_inicio, fecha_fin, estado
Pagos	Registra las transacciones de cobro de	id_pago, id_membresia, monto,

	una membresía	moneda, transaction_id, estado, fecha_pago
--	---------------	---

8. Script de base de datos

El script `docs/database/schema.sql` crea la base de datos FitControl y las cinco tablas del modelo (Usuarios, Clientes, Planes, Membresías, Pagos), con:

- Claves primarias autoincrementales (IDENTITY) en cada tabla.
- Claves foráneas: Membresías → Clientes y Planes; Pagos → Membresías.
- Restricciones CHECK para estados válidos (por ejemplo, estado IN ('Activo', 'Inactivo')) y para valores numéricos positivos (precio, monto).
- Índices sobre `id_cliente` y `estado` en Membresías, y sobre `id_membresia` en Pagos, para acelerar las consultas más frecuentes de la API (listar membresías por cliente o por estado).
- Datos de prueba (seeds): usuarios, clientes, planes, membresías y pagos de ejemplo, listos para probar la API sin partir de una base de datos vacía.

Archivo: `docs/database/schema.sql`

El código completo del script no se transcribe aquí; se entrega como archivo independiente, ejecutable directamente en SQL Server.

9. Estrategia de caché

9.1 Justificación

Se almacenarán en caché los datos que se consultan con mucha frecuencia y cambian poco: el catálogo de planes, los datos de un cliente por id, y los reportes agregados (ingresos, membresías vencidas), cuyo cálculo es más costoso que una lectura simple. El objetivo es reducir la carga sobre SQL Server y bajar el tiempo de respuesta de la API en las consultas repetidas.

9.2 Patrón seleccionado

Se aplica el patrón Cache-Aside (Lazy Loading): el API Backend es responsable de leer y actualizar el caché explícitamente. Ante una solicitud, primero se consulta Redis; si el dato existe (cache hit) se retorna de inmediato; si no existe (cache miss), se consulta la base de datos, se guarda el resultado en Redis con un tiempo de vida definido, y luego se responde al cliente.

9.3 Cache keys y TTL

Información	Cache key	TTL
Catálogo de planes	planes:all	1 hora
Cliente por id	cliente:{id_cliente}	10 min
Reporte de ingresos por rango	reporte:ingresos:{fecha_inicio}:{fecha_fin}	15 min
Membresías vencidas	reporte:vencidas	15 min

9.4 Política de invalidación

- Al crear un cliente: no se invalida nada (no existe aún caché por id para ese registro).
- Al actualizar un cliente: se elimina cliente:{id} para forzar una relectura fresca en la siguiente consulta.
- Al crear o actualizar un plan: se elimina planes:all, ya que afecta el catálogo completo.
- Al registrar una membresía o un pago: se eliminan reporte:ingresos:* y reporte:vencidas, porque esos datos agregados quedan desactualizados.

9.5 Flujo de caché

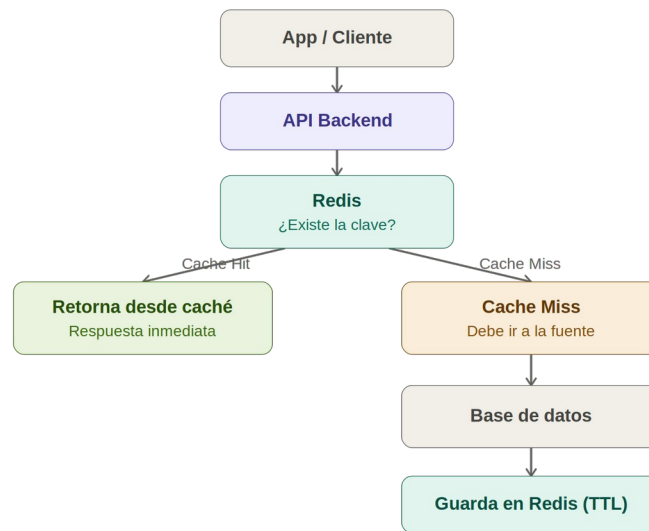


Diagrama 6 — Estrategia de caché: Cliente → API → Redis → Base de datos

Archivo: docs/cache/estrategia_cache.png

10. Matriz de trazabilidad

Necesidad / requisito	Solución	Evidencia
Problema de escalabilidad	Arquitectura Monolito Modular	Sección 4 — Análisis de alternativas
Acoplamiento del legado	Separación en capas (Controllers/Services/Repositorios)	Diagrama 1 — Arquitectura legado
Exposición de funcionalidades	API REST documentada	docs/api/openapi.yaml
Autenticación	JWT (Bearer token)	docs/api/openapi.yaml — securitySchemes
Arquitectura propuesta	Modelo C4	Diagramas 2, 3 y 4
Persistencia	Modelo de datos relacional	Diagrama 5 — DER
Creación de la base de datos	Script DDL	docs/database/schema.sql
Rendimiento en consultas frecuentes	Caché con Redis (Cache-Aside)	Diagrama 6 — Flujo de caché

11. Validación de la solución

11.1 Validación del contrato API

El archivo `openapi.yaml` fue validado sintácticamente (YAML válido, sin errores de indentación) y se comprobó visualmente cargándolo en `editor.swagger.io`, donde se confirmó que los dos endpoints, sus DTOs (`LoginRequest/Response`, `PaymentRequest/Response`), los códigos de estado y el esquema de seguridad `bearerAuth` se representan correctamente. Además, se implementó un servidor de referencia que respondió con los códigos 200, 201, 400 y 401 exactamente como especifica el contrato.

Evidencia: `docs/evidence/swagger_validacion.png`

11.2 Validación de arquitectura

Existe correspondencia directa entre los tres niveles C4 y la API: los actores del Nivel 1 usan los contenedores del Nivel 2 (App de escritorio / App Web-Mobile), que consumen el API Backend; y el API Backend del Nivel 2 se descompone en los componentes del Nivel 3 (Controllers, Services, Repositories), que son exactamente los que implementan los endpoints documentados en el OpenAPI.

11.3 Validación de persistencia

Cada DTO definido en el contrato API tiene su contraparte en el modelo de datos: `LoginRequest/Response` se valida contra la tabla Usuarios; los DTOs de Clientes y Membresías corresponden a las tablas Clientes, Planes y Membresías; y `PaymentRequest/Response` se persiste en la tabla Pagos, manteniendo consistencia entre API, DTO, modelo de datos y base de datos.

11.4 Validación de caché

La estrategia Cache-Aside es consistente con las operaciones de lectura y modificación: las lecturas frecuentes (planes, reportes) se sirven desde Redis con TTL, mientras que las mutaciones (crear membresía, procesar pago) disparan la invalidación de las claves de reporte afectadas, evitando que la API devuelva datos agregados desactualizados.

12. Conclusiones

- El problema principal del sistema legado no era funcional, sino arquitectónico: mezclar interfaz, lógica y acceso a datos en un mismo formulario impedía tanto el mantenimiento como la integración con sistemas externos.
- La arquitectura de Monolito Modular resuelve ese acoplamiento sin incurrir en la complejidad operativa de microservicios, que no se justifica para el volumen de un solo gimnasio.
- Los atributos de calidad más beneficiados son mantenibilidad (separación en capas) y seguridad (autenticación centralizada vía JWT), mientras que escalabilidad y disponibilidad mejoran de forma moderada gracias a la caché con Redis.
- Persisten riesgos a vigilar: la migración de contraseñas de texto plano a hashes, la implementación real del rate-limiting en la API, y la necesidad de definir un plan de respaldo (backup) para la base de datos, no cubierto en este informe.
- Antes de llevar la solución a producción debe definirse el monitoreo de la API (logs, métricas) y probarse la estrategia de caché bajo carga real, para calibrar los TTL propuestos.

13. Referencias

- OpenAPI Initiative — OpenAPI Specification 3.0.3.
- The C4 Model for Software Architecture — c4model.com.
- PlantUML / C4-PlantUML — github.com/plantuml-stdlib/C4-PlantUML.
- Redis Documentation — Cache-Aside pattern.
- Microsoft Docs — SQL Server Transact-SQL reference.
- JWT.io — JSON Web Token introduction.

Anexos

Espacio reservado para evidencias complementarias: capturas adicionales de Swagger UI, resultados de pruebas con Postman/curl, o cualquier material de soporte que no se haya incluido en el cuerpo principal del informe.